DEVOPS LAB / CHEAT SHEET

01 / Comprendre et expliquer - version du 19 octobre 2026

Objectif : reproduire localement une méthode DevOps inspirée de Sobrus : déployer, surveiller, sauvegarder et restaurer, avec un README et un runbook.

Le rôle de chaque outil

Docker Desktop + kind	Docker fournit le moteur Linux ; kind crée Kubernetes dans des conteneurs Docker.
Kubernetes + kubectl	Kubernetes orchestre les pods. kubectl permet de consulter et piloter le cluster.
Helm	Un chart regroupe les modèles Kubernetes ; values.yaml fournit les paramètres.
Registry local	Stocke les images. Windows : localhost:5001 ; réseau Docker : devops-lab-registry:5000.
GitHub + ArgoCD	Git contient l'état souhaité ; ArgoCD le synchronise avec le cluster (GitOps).
Traefik	Reçoit HTTPS, présente le certificat et route les requêtes vers le Service applicatif.
cert-manager	Émet et gère les certificats avec une autorité locale ; ne transporte pas le trafic web.
PostgreSQL + Redis	Base persistante et cache temporaire. Ils restent indépendants de l'application actuelle.
Prometheus + Grafana	Prometheus collecte et évalue les alertes ; Grafana affiche les métriques et les pods.
Alertmanager	Regroupe les alertes localement. Aucun email ou message externe configuré.

Les trois trajets à savoir dessiner

```
Utilisateur -> HTTPS :8443 -> tunnel vers Traefik :443
Traefik -> Service devops-demo :80 -> application Python :8080

Code -> build -> registry local -> image récupérée par Kubernetes
Configuration -> GitHub -> ArgoCD -> API Kubernetes -> déploiement

Exporteurs + kube-state-metrics -> collecte Prometheus -> Grafana
```

Accès utilisateur uniquement via Traefik. Le port direct 8081 est fermé. ArgoCD et cert-manager ne sont pas dans le chemin des requêtes web.

Vocabulaire Kubernetes

Pod / Deployment	Pod : exécute les conteneurs. Deployment : maintient les répliques d'une application.
Service / Ingress	Service : accès stable aux pods. Ingress : règle de routage utilisée par Traefik.
Namespace / Secret	Namespace : rangement logique. Secret : configuration sensible, pas un coffre-fort chiffré par défaut.
PVC / StatefulSet	PVC : demande de stockage. StatefulSet : identité stable pour un service comme PostgreSQL.
CronJob / Job	CronJob : planification. Job : exécution ponctuelle, par exemple une sauvegarde.

Les réponses qui font la différence

Synced = ressources suivies conformes à Git. Healthy = santé observée des ressources. Cela ne suffit pas à prouver l'accès du navigateur.

HTTPS = chiffrement + vérification du certificat. SAN doit contenir localhost. L'absence de CN n'est pas, seule, une erreur ; le navigateur doit approuver l'autorité locale.

Sauvegarde toutes les 5 min : attente PostgreSQL, pg_dump, archive et SHA256 sur un PVC de 2 Gi. Export Windows manuel. Restaurer = vérifier réellement la récupération.

Limites : un PC / un nœud, pas de haute disponibilité ni de sauvegarde hors site. L'application n'utilise ni PostgreSQL ni Redis. Les règles d'alerte ont des tests simulés.

COMMANDES / MÉMO OPÉRATEUR

02 / PowerShell - conserver un terminal ouvert par tunnel

1. Se placer dans le projet

```
Set-Location C:\Users\pc\Desktop\Devops-lab
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

À refaire dans chaque nouveau terminal qui lance des scripts. Docker Desktop doit être démarré.

2. Ouvrir l'application uniquement via Traefik

```
.\scripts\Open-Demo.ps1
# Navigateur : https://localhost:8443
```

Approbation explicite de l'autorité pour ton utilisateur Windows, si nécessaire :

```
.\scripts\Export-LabCA.ps1 -Trust
```

Puis fermer et rouvrir le navigateur. Ne pas contourner un avertissement : vérifier la confiance, le SAN et les dates.

3. Ouvrir Grafana et ArgoCD (un terminal chacun)

```
kubectl --context kind-devops-lab -n monitoring port-forward svc/grafana 3000:3000
# http://localhost:3000/d/devops-lab

kubectl --context kind-devops-lab -n argocd port-forward svc/argocd-server 8080:443
# https://localhost:8080 - certificat distinct de celui de l'application
```

Compte admin ; mots de passe locaux : docs/acces-interfaces.md. Ne pas les afficher pendant le partage d'écran.

4. Vérifier l'état et diagnostiquer

```
kubectl --context kind-devops-lab get nodes
kubectl --context kind-devops-lab get pods -A
kubectl --context kind-devops-lab -n argocd get applications
kubectl --context kind-devops-lab -n data get pods,jobs,pvc
.\scripts\Test-Deployment.ps1
.\scripts\Test-Monitoring.ps1
```

Diagnostic ciblé (remplacer NOM_DU_POD et choisir le bon namespace) :

```
kubectl --context kind-devops-lab -n data logs NOM_DU_POD --tail=50
kubectl --context kind-devops-lab -n data describe pod NOM_DU_POD
```

Running ≠ prêt : lire READY. Completed = job réussi. Error/Failed = échec. Alerte pending = condition présente ; firing = durée atteinte ; inactive = condition absente.

5. Sauvegarder et tester la restauration

```
.\scripts\Backup-Postgres.ps1
.\scripts\Test-Restore.ps1
```

La première commande exporte une archive vérifiée vers backups/. La seconde ajoute une ligne fictive et conserve une nouvelle instance/PVC ; la source reste active. Ne pas relancer inutilement.

6. Publier une version puis revenir en arrière

```
git status --short
.\scripts\Publish-Demo.ps1
$deployCommit = git rev-parse HEAD
.\scripts\Test-Deployment.ps1
# Retour arrière du commit de publication, dans le même terminal :
git revert --no-edit $deployCommit
git push origin main
.\scripts\Test-Deployment.ps1
```

Avant publication : main propre et synchronisée, droits GitHub et registry local disponibles. ArgoCD suit Git : un rollout undo manuel serait corrigé par selfHeal.

Pour la présentation : préparer les interfaces et les rapports. Montrer les preuves ; ne pas réinstaller le lab en direct. Ne supprimer aucun cluster, volume ou sauvegarde sans décision explicite.